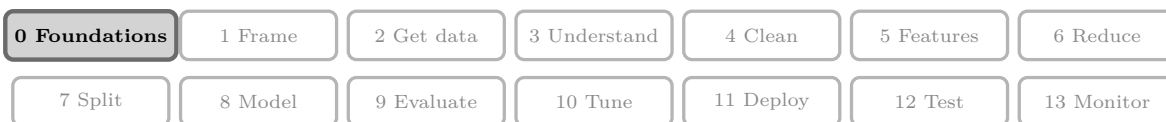


Computing the SVD by Hand

Contents

1. Overview	2
2. Making one orthogonal matrix disappear	3
2.1 $A^T A$ gives V and the singular values	4
2.2 Why this always works	7
2.3 AA^T gives U , with the same numbers	8
3. The recipe, worked on a 2×2 matrix	9
4. The sign trap	13
5. A matrix of rank 1	15
6. The four subspaces	18
7. A matrix that is not square	21
8. How computers compute the SVD	24
9. Summary	27
10. Sources	27
11. Key terms	28

Where this fits: Pipeline map step 0 (Foundations). See the [Course map](#).



- **Builds on:** Linear transformations and matrices ([Note MA-053](#)); Eigenvectors and eigenvalues ([Note MA-056](#)).
- **Leads to:** Low-rank approximation (truncated SVD) ([Note MA-059](#)); Latent semantic analysis ([Note MA-060](#)); Moore-Penrose pseudo-inverse ([Note MA-060](#)).
- **Compare with:** Eigenvectors and eigenvalues ([Note MA-056](#)).

1. Overview

Key point: A matrix stretches some directions more than others. To find the SVD by hand, we multiply the matrix by its own transpose. That new matrix keeps the stretching but forgets the final turn, so its eigenvectors are the directions of most and least stretch, and its eigenvalues are the squared stretches.

The first SVD Note, the [SVD geometry Note](#), showed what $A = U\Sigma V^T$ means: turn, stretch, turn. The three factors are:

- V , whose columns $\mathbf{v}_1, \mathbf{v}_2$ are the input directions (the **right singular vectors**, G-1692);
- Σ , the **diagonal matrix** (G-601) of stretches σ_1, σ_2 (the **singular values**, G-1812);
- U , whose columns $\mathbf{u}_1, \mathbf{u}_2$ are the output directions (the **left singular vectors**, G-1079).

It gave the factors of

$$A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$$

without saying where they came from. This Note finds them, with a four-step recipe:

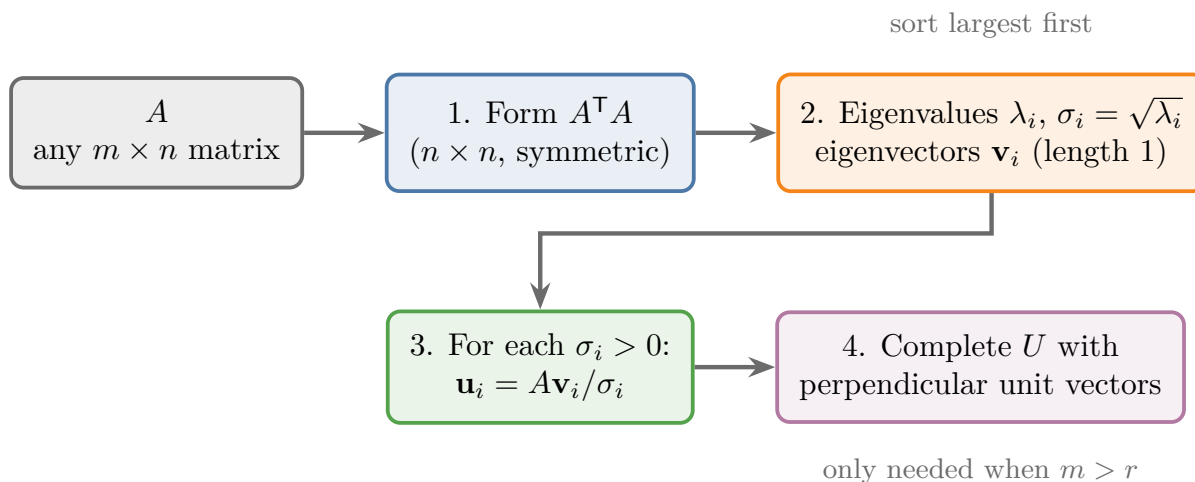


Figure 1: The four steps of computing an SVD by hand

Figure 1 is the plan for the whole Note. Read it top to bottom: form a new symmetric matrix, take its eigenvectors and eigenvalues, turn the eigenvalues into stretches, then get the output directions from the input directions.

This Note covers:

- why $A^T A$ and AA^T hold the answer (Section 2);
- the four-step recipe of Figure 1, worked on A (Section 3);
- a sign trap, and how computing each $\mathbf{u}_i$ from its $\mathbf{v}_i$ avoids it (Section 4);
- a matrix of rank 1, where a singular value is 0 (Section 5);
- the four subspaces that the SVD gives bases for (Section 6);
- a matrix that is not square (Section 7);
- why computers do not use this recipe (Section 8).

Finding eigenvalues and eigenvectors by hand, with the determinant, is taught in the [eigenvectors and eigenvalues Note](#); we use it here without repeating it.

2. Making one orthogonal matrix disappear

Key point: The SVD has two unknown turning matrices, U and V . In $A^T A$, the turn U cancels out, leaving a symmetric matrix whose eigenvectors and eigenvalues we can find by hand.

2.1 $A^T A$ gives V and the singular values

Key point: The directions in which A stretches most and least are the eigenvectors of $A^T A$; the squared stretches are its eigenvalues.

The idea in plain words. Feed A every arrow of length 1 and measure how long each output is. Some input directions get stretched a lot, others only a little. The longest and shortest stretches, and the directions that get them, are exactly the singular values and the right singular vectors.

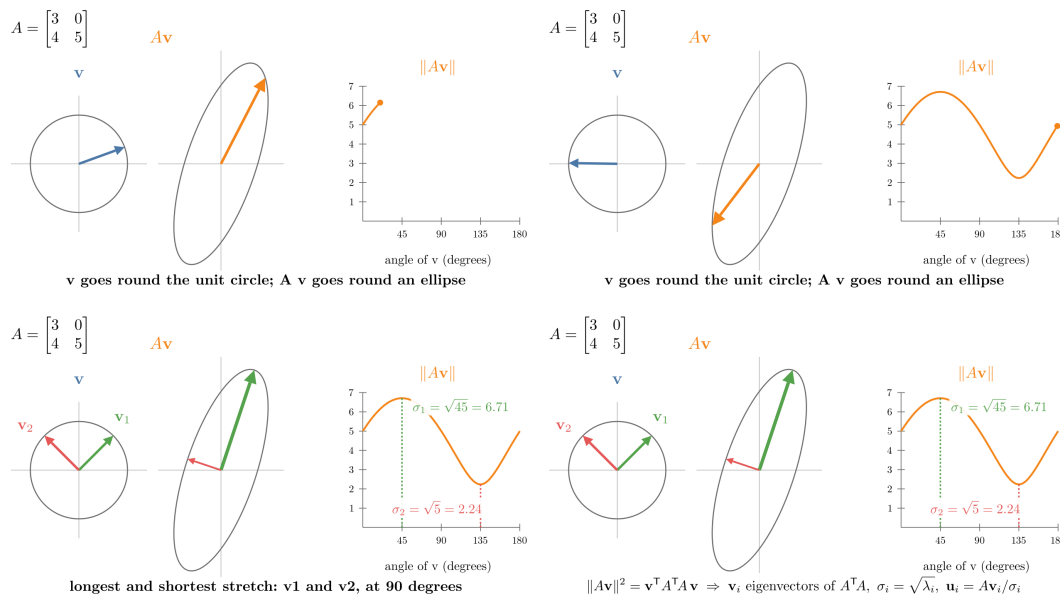


Figure 2: A unit vector v sweeps the circle and Av sweeps an ellipse; the length of Av peaks at $\sigma_1 = \sqrt{45}$ along v_1 and bottoms out at $\sigma_2 = \sqrt{5}$ along v_2 , at 90° to v_1

Figure 2 does this for A . A unit arrow v goes round the circle (left), its output Av goes round an ellipse (middle), and the graph (right) tracks the length of Av . Watch the peak and the trough:

- the longest stretch, 6.71, comes at the direction v_1 ;
- the shortest, 2.24, comes at v_2 , at a right angle to v_1 .

The two directions, each scaled to length 1:

$$v_1 = [1, 1]/\sqrt{2}$$

$$v_2 = [-1, 1]/\sqrt{2}$$

The question is how to find these two directions without sweeping every arrow.

The trick on numbers. The squared length of an output is the output dotted with itself, and that can be regrouped:

$$\|A\mathbf{v}\|^2 = (A\mathbf{v})^T(A\mathbf{v})$$

$$= \mathbf{v}^T(A^T A)\mathbf{v}$$

So the matrix $A^T A$ measures squared stretch. Here A^T is the **transpose** (G-2012) of A : the same numbers with rows and columns swapped. Compute $A^T A$ entry by entry:

$$A^T = \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix}$$

Entry of $A^T A$	Row of A^T times column of A	Value
row 1, column 1	$3 \times 3 + 4 \times 4$	25
row 1, column 2	$3 \times 0 + 4 \times 5$	20
row 2, column 1	$0 \times 3 + 5 \times 4$	20
row 2, column 2	$0 \times 0 + 5 \times 5$	25

$$A^T A = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$$

Now apply $A^T A$ to the peak direction $\mathbf{v}_1$ of Figure 2 (the vector $[1, 1]$, scaled to length 1):

$$\begin{aligned} & A^T A \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 25 + 20 \\ 20 + 25 \end{bmatrix} \\ &= \begin{bmatrix} 45 \\ 45 \end{bmatrix} \\ &= 45 \begin{bmatrix} 1 \\ 1 \end{bmatrix} \end{aligned}$$

The direction did not change; it was only scaled by 45. So $\mathbf{v}_1$ is an **eigenvector** (G-666) of $A^T A$ with **eigenvalue** (G-665) 45. And

$$\sqrt{45} = 6.71$$

is the peak stretch of Figure 2. The same check on the trough direction $[-1, 1]$:

$$\begin{aligned} & A^T A \begin{bmatrix} -1 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} -25 + 20 \\ -20 + 25 \end{bmatrix} \\ &= \begin{bmatrix} -5 \\ 5 \end{bmatrix} \\ &= 5 \begin{bmatrix} -1 \\ 1 \end{bmatrix} \\ &\sqrt{5} = 2.24 \end{aligned}$$

the trough stretch.

Why it works: the algebra, one step per line. Start from $A = U\Sigma V^T$. The transpose of a product is the product of the transposes in reverse order, and $U^T U = I$, the **identity matrix** (G-915), because U is an **orthogonal matrix** (G-1407) (see the [SVD geometry Note](#), section 2.5):

$$A^T = (U\Sigma V^T)^T = V\Sigma^T U^T$$

$$A^T A = V\Sigma^T U^T U\Sigma V^T$$

$$= V\Sigma^T I \Sigma V^T$$

$$= V \begin{bmatrix} \sigma_1^2 & & \\ & \ddots & \\ & & \sigma_n^2 \end{bmatrix} V^T$$

The right side is exactly an eigen-decomposition PDP^{-1} (see the [eigenvectors and eigenvalues Note](#), section 6.2), with $P = V$ and $P^{-1} = V^T$ (Strang §7.2; MML §4.5.2). So:

- the right singular vectors $\mathbf{v}_i$ are the eigenvectors of $A^T A$;
- the squared singular values σ_i^2 are its eigenvalues λ_i , so $\sigma_i = \sqrt{\lambda_i}$.

Check: for A , the eigenvalues 45 and 5 give $\sigma_1 = 6.71$ and $\sigma_2 = 2.24$, the peak and trough of Figure 2.

2.2 Why this always works

Key point: $A^T A$ is symmetric with eigenvalues of at least 0, so it always has perpendicular eigenvectors and real square roots.

A itself may have no useful eigenvectors, but $A^T A$ always does, for two reasons.

It is symmetric. The matrix $A^T A$ of Section 2.1 reads the same across its diagonal: 20 above and 20 below. That always happens, because its transpose is itself:

$$(A^T A)^T = A^T (A^T)^T = A^T A$$

A **symmetric matrix** (G-1932) always has a full set of perpendicular eigenvectors (the same fact that makes PCA work on a covariance matrix, see the [PCA step by step Note](#), section 4.3). So V can always be built as an orthogonal matrix. Check: the dot product of $[1, 1]$ and $[-1, 1]$ is

$$-1 + 1 = 0$$

so they are perpendicular.

Its eigenvalues are never negative. Take a unit eigenvector $\mathbf{v}$ with eigenvalue λ . Then, one step per line:

$$\lambda = \lambda \mathbf{v}^T \mathbf{v} = \mathbf{v}^T (A^T A \mathbf{v})$$

$$= (A \mathbf{v})^T (A \mathbf{v})$$

$$= \|A \mathbf{v}\|^2 \geq 0$$

Here $\|A \mathbf{v}\|$ is the length of the output. Check with $\mathbf{v}_1$: the output $A \mathbf{v}_1$ is the vector $[3, 9]$ scaled by $1/\sqrt{2}$, and its squared length is the eigenvalue:

$$(9 + 81)/2 = 45$$

So the square root $\sigma = \sqrt{\lambda}$ is a real number of at least 0, and it is the length of $A \mathbf{v}$, as in Figure 2.

A symmetric matrix with no negative eigenvalues is **positive semi-definite** (G-1532). So $A^T A$ is symmetric and positive semi-definite for every matrix A of any shape.

2.3 AA^T gives U , with the same numbers

Key point: Multiplying in the other order, AA^T , cancels V instead and keeps U ; its non-zero eigenvalues are the same squared stretches.

In plain words. $A^T A$ looks at the input side and finds the $\mathbf{v}$'s. Swapping the order looks at the output side and finds the $\mathbf{u}$'s.

On numbers. For A :

$$\begin{aligned} AA^T &= \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix} \begin{bmatrix} 3 & 4 \\ 0 & 5 \end{bmatrix} \\ &= \begin{bmatrix} 9 & 12 \\ 12 & 41 \end{bmatrix} \end{aligned}$$

a different matrix from $A^T A$. Its eigenvalues still add up to the diagonal sum and multiply to the determinant:

$$9 + 41 = 50 = 45 + 5$$

$$9 \times 41 - 12 \times 12 = 369 - 144 = 225 = 45 \times 5$$

so they are 45 and 5 again.

The algebra, by the same steps as Section 2.1, using $V^T V = I$:

$$\begin{aligned} AA^T &= U \Sigma V^T V \Sigma^T U^T \\ &= U \Sigma \Sigma^T U^T \end{aligned}$$

So the left singular vectors $\mathbf{u}_i$ are the eigenvectors of AA^T , and its non-zero eigenvalues are the same σ_i^2 . The match of 45 and 5 is no accident: AB and BA always share their non-zero eigenvalues.

3. The recipe, worked on a 2×2 matrix

Key point: Find the eigenvectors and eigenvalues of $A^T A$, take square roots, then get each $\mathbf{u}_i$ by applying A to $\mathbf{v}_i$ and shrinking the result to length 1.

Figure 1 shows the four steps. We follow them for A with rows $[3, 0]$ and $[4, 5]$.

Step 1: form $A^T A$. As in Section 2.1:

$$A^T A = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$$

Step 2: eigenvalues and eigenvectors of $A^T A$.

In words: an eigenvalue λ is a number for which $A^T A - \lambda I$ (subtract λ from the diagonal) squashes some arrow to zero; such a matrix has determinant 0.

On numbers:

$$\det \begin{bmatrix} 25 - \lambda & 20 \\ 20 & 25 - \lambda \end{bmatrix} = (25 - \lambda)^2 - 20 \times 20$$

$$(25 - \lambda)^2 - 400 = 0$$

$$25 - \lambda = 20$$

or

$$25 - \lambda = -20$$

$$\lambda_1 = 45$$

$$\lambda_2 = 5$$

The eigenvalues are listed largest first.

$$\sigma_1 = \sqrt{45} = 3\sqrt{5} \approx 6.708$$

$$\sigma_2 = \sqrt{5} \approx 2.236$$

For each eigenvalue, find the line of arrows it squashes to zero:

λ	$A^T A - \lambda I$	sends $[x, y]$ to zero when
45	rows $[-20, 20], [20, -20]$	$x = y$
5	rows $[20, 20], [20, 20]$	$y = -x$

Scaled to length 1 (divide by the length of $[1, 1]$):

$$\sqrt{1^2 + 1^2} = \sqrt{2}$$

$$\mathbf{v}_1$$

$$= \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\mathbf{v}_2$$

$$= \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

The two vectors are perpendicular, as Section 2.2 promised.

The formula:

$$\det(A^T A - \lambda I) = 0$$

$$\sigma_i = \sqrt{\lambda_i}$$

Step 3: get each $\mathbf{u}_i$ from its $\mathbf{v}_i$.

In words: apply A to each right singular vector; the output points along an axis of the ellipse and has length σ_i . Shrink it back to length 1.

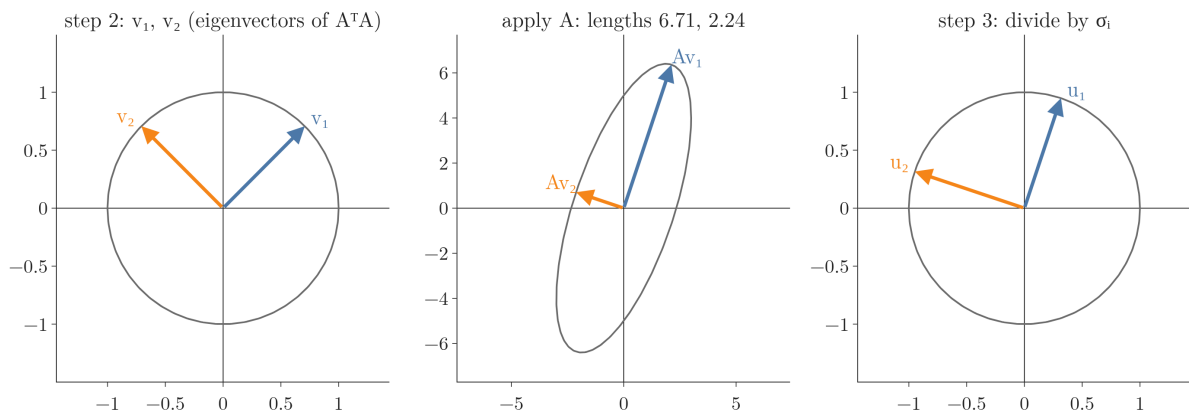


Figure 3: Steps 2 and 3 for A : the eigenvectors $\mathbf{v}_1, \mathbf{v}_2$ of $A^T A$ (left), their images under A with lengths σ_1, σ_2 (middle), and the same images shrunk back to length 1, which are $\mathbf{u}_1, \mathbf{u}_2$ (right)

In Figure 3, watch the middle panel: $A\mathbf{v}_1$ and $A\mathbf{v}_2$ already point along the ellipse's axes, so dividing by their lengths only shrinks them back onto the circle (right).

On numbers:

$$\begin{aligned}
 & A\mathbf{v}_1 \\
 &= \frac{1}{\sqrt{2}} \begin{bmatrix} 3 \times 1 + 0 \times 1 \\ 4 \times 1 + 5 \times 1 \end{bmatrix} \\
 &= \frac{1}{\sqrt{2}} \begin{bmatrix} 3 \\ 9 \end{bmatrix} \\
 & \mathbf{u}_1 \\
 &= \frac{1}{3\sqrt{5}} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 3 \\ 9 \end{bmatrix} \\
 &= \frac{1}{\sqrt{10}} \begin{bmatrix} 1 \\ 3 \end{bmatrix} \\
 & A\mathbf{v}_2 \\
 &= \frac{1}{\sqrt{2}} \begin{bmatrix} 3 \times (-1) + 0 \times 1 \\ 4 \times (-1) + 5 \times 1 \end{bmatrix}
 \end{aligned}$$

$$= \frac{1}{\sqrt{2}} \begin{bmatrix} -3 \\ 1 \end{bmatrix}$$

$$\mathbf{u}_2$$

$$= \frac{1}{\sqrt{5}} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} -3 \\ 1 \end{bmatrix}$$

$$= \frac{1}{\sqrt{10}} \begin{bmatrix} -3 \\ 1 \end{bmatrix}$$

The formula: the step is the **singular value equation** (G-1814) $A\mathbf{v}_i = \sigma_i \mathbf{u}_i$ solved for $\mathbf{u}_i$:

$$\mathbf{u}_i = \frac{1}{\sigma_i} A\mathbf{v}_i$$

The $\mathbf{u}$'s come out perpendicular without any extra work. Check on numbers: the dot product of $[1, 3]$ and $[-3, 1]$ is

$$-3 + 3 = 0$$

In symbols, one step per line:

$$(A\mathbf{v}_1)^\top (A\mathbf{v}_2) = \mathbf{v}_1^\top (A^\top A\mathbf{v}_2)$$

$$= \mathbf{v}_1^\top (\lambda_2 \mathbf{v}_2) = \lambda_2 \mathbf{v}_1^\top \mathbf{v}_2 = 0$$

Step 4: complete U . Here there are already two $\mathbf{u}$'s for a 2×2 matrix, so nothing is missing. Sections 5 and 7 need this step.

Check. Multiplying the factors back together, one product at a time:

$$U\Sigma$$

$$= \frac{1}{\sqrt{10}} \begin{bmatrix} 1 & -3 \\ 3 & 1 \end{bmatrix}$$

$$\cdot \begin{bmatrix} 3\sqrt{5} & 0 \\ 0 & \sqrt{5} \end{bmatrix}$$

$$= \frac{1}{\sqrt{2}} \begin{bmatrix} 3 & -3 \\ 9 & 1 \end{bmatrix}$$

$$U\Sigma V^T$$

$$= \frac{1}{\sqrt{2}} \begin{bmatrix} 3 & -3 \\ 9 & 1 \end{bmatrix}$$

$$\cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$$

$$= \frac{1}{2} \begin{bmatrix} 6 & 0 \\ 8 & 10 \end{bmatrix}$$

$$= \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$$

$$= A$$

Python: Checking each step with NumPy.

```
import numpy as np
```

```
A = np.array([[3, 0], [4, 5]])
```

```
lam, V = np.linalg.eigh(A.T @ A)    # eigh: for symmetric
```

```
lam, V = lam[::-1], V[:, ::-1]    # largest first
```

```
sigma = np.sqrt(lam)              # [6.708, 2.236]
```

```
U = A @ V / sigma                 # column i = A v_i / sigma_i
```

```
U @ np.diag(sigma) @ V.T         # gives back A
```

`A @ V / sigma` divides column i of AV by σ_i : NumPy matches the 1D array `sigma` with the columns.

4. The sign trap

Key point: Finding U and V separately as eigenvectors can pick mismatched signs and give a wrong product; computing $\mathbf{u}_i = A\mathbf{v}_i/\sigma_i$ always matches them.

Section 2.3 suggests a shortcut: take the $\mathbf{u}$'s as the eigenvectors of AA^T , found on their own. For A , AA^T has rows $[9, 12]$ and $[12, 41]$, with eigenvalues 45 and 5. Its eigenvectors are the lines through $[1, 3]$ and through $[3, -1]$.

Every non-zero multiple of an eigenvector is an eigenvector too, so $\frac{1}{\sqrt{10}}[3, -1]$ is as correct an eigenvector as $\frac{1}{\sqrt{10}}[-3, 1]$. But it is the negative of the $\mathbf{u}_2$ that belongs with our $\mathbf{v}_2$. Using it gives

$$\frac{1}{\sqrt{10}} \begin{bmatrix} 1 & 3 \\ 3 & -1 \end{bmatrix}$$

$$\cdot \begin{bmatrix} 3\sqrt{5} & 0 \\ 0 & \sqrt{5} \end{bmatrix}$$

$$\cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 3 \\ 5 & 4 \end{bmatrix}$$

$$\neq A$$

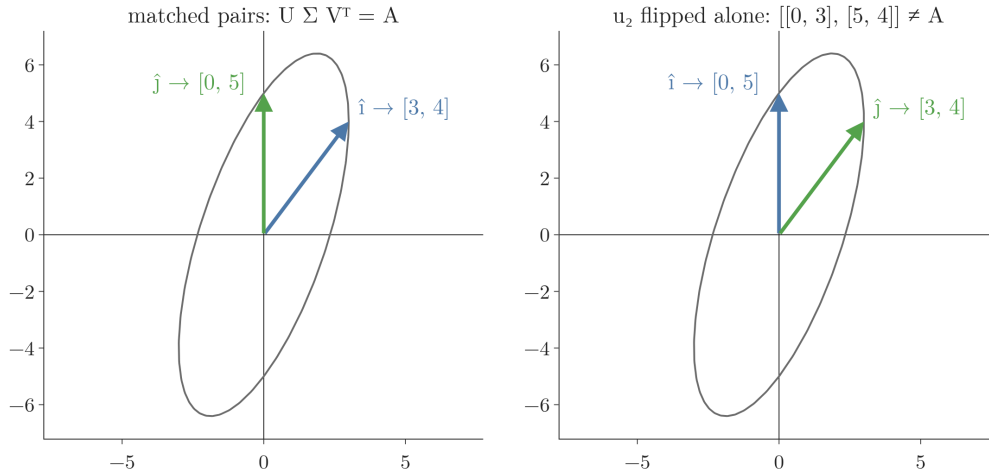


Figure 4: Left: the matched factors rebuild A . Right: the same factors with only $\mathbf{u}_2$ flipped. The ellipse is identical, but $\hat{\mathbf{i}}$ and $\hat{\mathbf{j}}$ land in swapped places

Each piece is a valid eigenvector, yet the product is a different matrix. Figure 4 shows why the mistake is easy to miss: both products have the singular values 6.71 and 2.24, so they draw the same ellipse; only where each input lands has changed. The pairs $(\mathbf{u}_i, \mathbf{v}_i)$ must match: flipping both signs is allowed, flipping one is not (see the [SVD geometry Note](#), section 5.1). The eigenvalue problem for AA^T knows nothing about V , so it cannot keep the pairs matched.

The fix is Step 3: compute $\mathbf{u}_i = A\mathbf{v}_i/\sigma_i$ from the $\mathbf{v}_i$ already chosen. Then $A\mathbf{v}_i = \sigma_i\mathbf{u}_i$ holds by construction, whatever sign each $\mathbf{v}_i$ was given.

5. A matrix of rank 1

Key point: A rank-1 matrix has one non-zero singular value; the zero one has no $\mathbf{u}$ from $A\mathbf{v}/\sigma$, so we complete U with a perpendicular unit vector.

Some matrices squash the plane flat. Take

$$C = \begin{bmatrix} 2 & 1 \\ 4 & 2 \end{bmatrix}$$

Its columns $[2, 4]$ and $[1, 2]$ both lie on the line through $[1, 2]$. Every output $C\mathbf{x}$ is a mix of the two columns, so every output lands on that one line: C squashes the whole plane onto it.

- The set of all outputs is the **column space** (G-414) of C : here, the line through $[1, 2]$.
- The number of dimensions of the column space is the **rank** (G-1627) of the matrix: here 1. (The second row is twice the first, the algebraic sign of the same fact.)
- While the plane squashes onto a line, a whole other line of inputs, the one through $[-1, 2]$, lands on the origin. The set of inputs that land on $\mathbf{0}$ is the **null space** (G-1362).

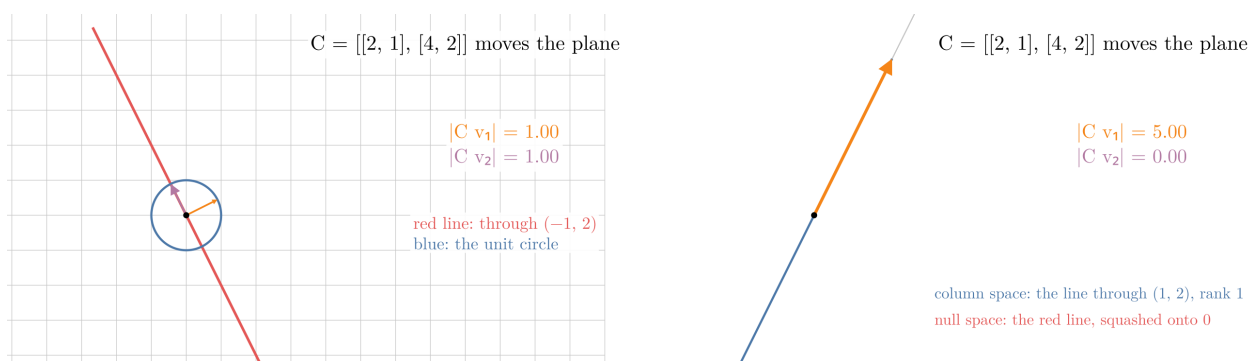


Figure 5: The rank-1 matrix C moves the grid: the whole plane, and the unit circle (blue) with it, squashes onto the line through $[1, 2]$, while every point of the red line through $[-1, 2]$ lands on the origin. $\mathbf{v}_1$ (orange) ends at length 5 and $\mathbf{v}_2$ (purple) at length 0. Idea after 3Blue1Brown, “Inverse matrices, column space and null space | Chapter 7, Essence of linear algebra”

In Figure 5, watch the red line while the grid moves: it shrinks towards the origin and vanishes, while the circle flattens into a segment. The two readouts end at 5 and 0, the two singular values the recipe now computes.

Step 1.

$$C^T C = \begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 2 & 1 \\ 4 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} 20 & 10 \\ 10 & 5 \end{bmatrix}$$

Step 2. The determinant and the diagonal sum give the two eigenvalues:

$$\det = 20 \times 5 - 10 \times 10 = 0$$

$$\implies \lambda_2 = 0$$

$$\lambda_1 + \lambda_2 = 20 + 5 = 25$$

$$\implies \lambda_1 = 25$$

$$\sigma_1 = \sqrt{25} = 5$$

$$\sigma_2 = 0$$

The eigenvectors are

$$\mathbf{v}_1$$

$$= \frac{1}{\sqrt{5}} \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

$$\mathbf{v}_2$$

$$= \frac{1}{\sqrt{5}} \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

$\mathbf{v}_2$ is the direction C squishes to nothing:

$$C \begin{bmatrix} -1 \\ 2 \end{bmatrix}$$

$$= \begin{bmatrix} 2 \times (-1) + 1 \times 2 \\ 4 \times (-1) + 2 \times 2 \end{bmatrix}$$

$$= \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

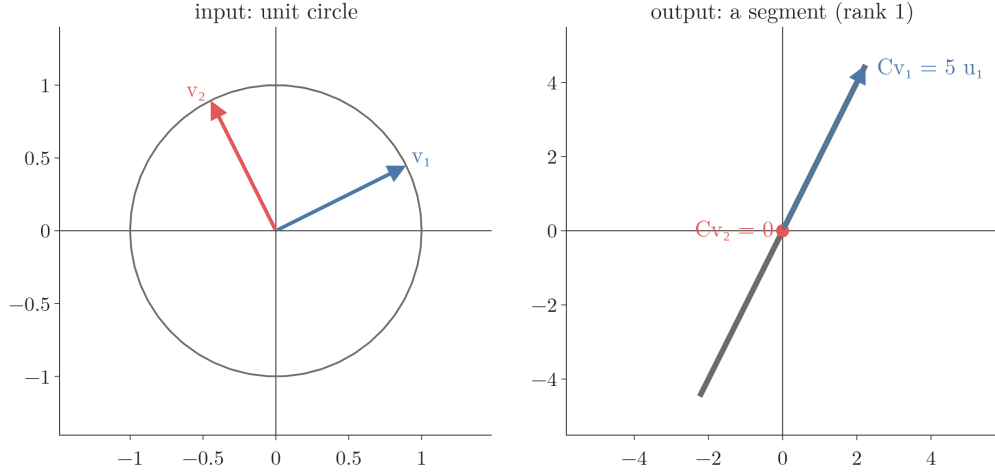


Figure 6: C flattens the unit circle onto a segment of the line through $[1, 2]$: $\mathbf{v}_1$ lands at length 5, $\mathbf{v}_2$ lands on the origin

In Figure 6, the ellipse of the 2×2 case has collapsed: its short axis σ_2 is 0, so the whole circle lands on one line, reaching 5 on each side of the origin.

Step 3. Only σ_1 is non-zero:

$$\mathbf{u}_1 = \frac{1}{5}C\mathbf{v}_1$$

$$= \frac{1}{5\sqrt{5}} \begin{bmatrix} 5 \\ 10 \end{bmatrix}$$

$$= \frac{1}{\sqrt{5}} \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

Step 4. Computing $\mathbf{u}_2$ as $C\mathbf{v}_2/\sigma_2$ would divide zero by zero. Any unit vector perpendicular to $\mathbf{u}_1$ completes U to an orthogonal matrix; we take:

$$\mathbf{u}_2 = \frac{1}{\sqrt{5}}[-2, 1]$$

This $\mathbf{u}_2$ is multiplied by $\sigma_2 = 0$, so it never affects the product.

Result.

$$\begin{aligned}
C &= \frac{1}{\sqrt{5}} \begin{bmatrix} 1 & -2 \\ 2 & 1 \end{bmatrix} \\
&\quad \cdot \begin{bmatrix} 5 & 0 \\ 0 & 0 \end{bmatrix} \\
&\quad \cdot \frac{1}{\sqrt{5}} \begin{bmatrix} 2 & 1 \\ -1 & 2 \end{bmatrix}
\end{aligned}$$

Only the first column of U and the first row of V^T meet a non-zero number, so the product shrinks to

$$\begin{aligned}
C &= 5 \cdot \frac{1}{\sqrt{5}} \begin{bmatrix} 1 \\ 2 \end{bmatrix} \cdot \frac{1}{\sqrt{5}} \begin{bmatrix} 2 & 1 \end{bmatrix} \\
&= \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} 2 & 1 \end{bmatrix} \\
&= \begin{bmatrix} 2 & 1 \\ 4 & 2 \end{bmatrix}
\end{aligned}$$

A column times a row is a whole matrix of rank 1. The [low-rank approximation Note](#) builds every matrix out of such pieces.

Python: NumPy reports the zero singular value.

```
C = np.array([[2, 1], [4, 2]])
```

```
np.linalg.svd(C)[1]      # [5., 0.]
```

```
np.linalg.matrix_rank(C) # 1
```

`matrix_rank` from the [linear combinations, span and basis Note](#) works exactly this way: it counts the singular values that are not (almost) zero.

6. The four subspaces

Key point: Every matrix splits its inputs into two perpendicular parts, the directions it uses and the directions it squashes to zero, and splits its outputs into the directions it reaches and the directions it never reaches. The SVD gives perpendicular bases for all four.

The idea in plain words, on C . Look again at the rank-1 matrix C of Figures 5 and 6.

- On the input side, there are two perpendicular lines. Along the line through $[2, 1]$, C does all its work: it stretches $\mathbf{v}_1$ by 5. Along the line through $[-1, 2]$, C squashes everything to zero.
- On the output side, there are also two perpendicular lines. Every output lands on the line through $[1, 2]$. The perpendicular line through $[-2, 1]$ is never reached by any output.

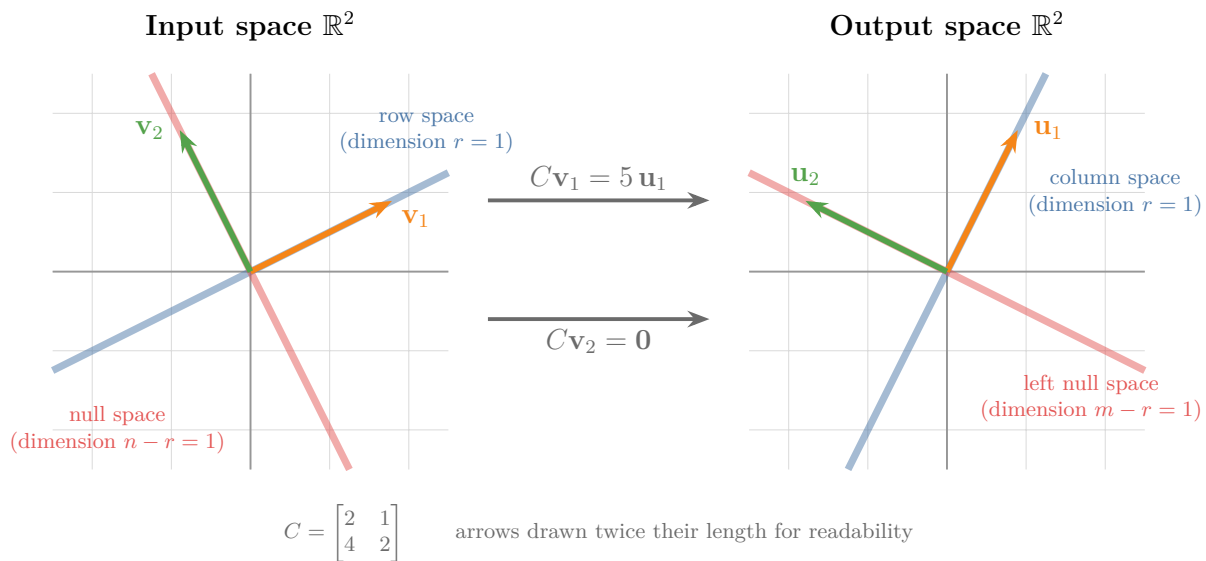


Figure 7: The SVD of C gives a basis for each of its four subspaces: input side (left) split into the row space and the null space, output side (right) split into the column space and the left null space

Figure 7 draws these four lines. On the left is the input plane, on the right the output plane; the titles write each plane as $\mathbb{R}^2$, the space of all pairs of numbers such as $(2, 1)$. The arrow $\mathbf{v}_1$ on the left is carried to the line of $\mathbf{u}_1$ on the right; the arrow $\mathbf{v}_2$ goes to the origin.

Worked check on numbers. One line per subspace:

$$\begin{aligned}
 & C \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\
 &= \begin{bmatrix} 5 \\ 10 \end{bmatrix} \\
 &= 5 \begin{bmatrix} 1 \\ 2 \end{bmatrix}
 \end{aligned}$$

The used direction lands on the output line.

$$C \begin{bmatrix} -1 \\ 2 \end{bmatrix} \\ = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

The squashed direction goes to zero.

$$[1, 2] \cdot [-2, 1] = -2 + 2 \\ = 0$$

The unreached direction is perpendicular to the output line.

The standard terms. Together these are the **four fundamental subspaces** (G-799) of a matrix (Strang §7.2). A **subspace** here is a line, plane or higher flat set through the origin. We write $\mathbb{R}^n$ for the space of all lists of n numbers: $\mathbb{R}^2$ holds pairs such as $(2, 1)$, $\mathbb{R}^3$ triples such as $(2, 1, 1)$. An $m \times n$ matrix (m rows, n columns) takes inputs from $\mathbb{R}^n$ and gives outputs in $\mathbb{R}^m$; for C , $m = n = 2$. For a matrix of rank r (there are r non-zero singular values):

Subspace	Where	Basis from the SVD	Dimension	For C
Column space (G-414) (span of the columns: every output)	output $\mathbb{R}^m$	$\mathbf{u}_1, \dots, \mathbf{u}_r$	r	line through $[1, 2]$
Null space (G-1362) (vectors sent to $\mathbf{0}$)	input $\mathbb{R}^n$	$\mathbf{v}_{r+1}, \dots, \mathbf{v}_n$	$n - r$	line through $[-1, 2]$
Row space (G-1713) (span of the rows)	input $\mathbb{R}^n$	$\mathbf{v}_1, \dots, \mathbf{v}_r$	r	line through $[2, 1]$
Left null space (G-1078) (outputs never reached, perpendicular to them)	output $\mathbb{R}^m$	$\mathbf{u}_{r+1}, \dots, \mathbf{u}_m$	$m - r$	line through $[-2, 1]$

Check: for C , $r = 1$, so each subspace has dimension 1 (a line), as the last column says.

The first two are the ones Figure 5 showed moving: the column space is where every output lands (the **span** (G-1838) of the columns, from the [linear transformations and matrices Note](#), section 6), and the null space is everything the matrix squishes to the origin. The other two are their partners: the row space is the input directions perpendicular to the null space, and the left null space is the output directions perpendicular to the column space.

What makes the SVD's bases special is that they line up in pairs: A sends $\mathbf{v}_1$ to a multiple of $\mathbf{u}_1$, $\mathbf{v}_2$ to a multiple of $\mathbf{u}_2$, and so on, with no mixing. The null-space $\mathbf{v}$'s go to zero, which is where the zeros on the diagonal of Σ come from. Any method of making perpendicular bases could give bases of the four subspaces; only the SVD's bases also make the matrix diagonal.

7. A matrix that is not square

Key point: A tall matrix turns short lists into long lists, so its outputs cannot fill the bigger space. $A^\top A$ is the small matrix to work with, and the missing columns of U point in the directions the outputs never reach.

The idea in plain words. Take the 3×2 matrix B of the [SVD geometry Note](#) (section 6):

$$B = \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$$

It takes an input of 2 numbers and gives an output of 3 numbers. For example:

$$\begin{aligned} B \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 2 + 1 \\ 0 + 1 \\ 2 + 0 \end{bmatrix} \\ &= \begin{bmatrix} 3 \\ 1 \\ 2 \end{bmatrix} \end{aligned}$$

Every output is a mix of B 's two columns, $[1, 0, 1]$ and $[1, 1, 0]$. Two arrows in 3D span a flat sheet through the origin, so all outputs lie on that sheet. One direction, sticking straight out of the sheet, is never reached.

outputs of B fill a plane in 3-D

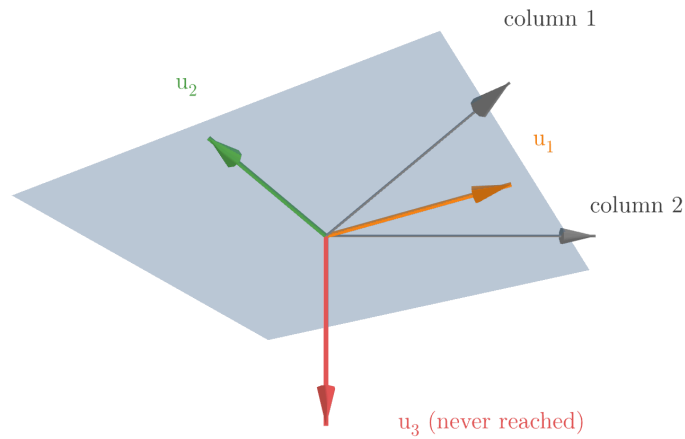


Figure 8: The outputs of B fill a plane in 3-D (blue): every output is a mix of B's two columns (grey). The left singular vectors $\mathbf{u}_1$ (orange) and $\mathbf{u}_2$ (green) lie in the plane; $\mathbf{u}_3$ (red) sticks out of it at a right angle and is never reached

Figure 8 shows the sheet. The recipe will find $\mathbf{u}_1$ and $\mathbf{u}_2$ inside the sheet (Step 3), and Step 4 adds $\mathbf{u}_3$, the red arrow at a right angle to it.

Steps 1 and 2. $B^T B$ is only 2×2 :

$$\begin{aligned}
 & B^T B \\
 &= \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} \\
 &\quad \cdot \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \\
 &= \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \\
 &(2 - \lambda)^2 - 1 = 0
 \end{aligned}$$

$$2 - \lambda = 1$$

or

$$2 - \lambda = -1$$

$$\lambda_1 = 3$$

$$\lambda_2 = 1$$

So the singular values are:

$$\sigma_1 = \sqrt{3} \approx 1.732$$

$$\sigma_2 = 1$$

with these directions:

$$\mathbf{v}_1$$

$$= \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

$$\mathbf{v}_2$$

$$= \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Step 3.

$$\mathbf{u}_1 = \frac{1}{\sqrt{3}} B \mathbf{v}_1$$

$$= \frac{1}{\sqrt{3}} \cdot \frac{1}{\sqrt{2}} \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}$$

$$= \frac{1}{\sqrt{6}} \begin{bmatrix} 2 \\ 1 \\ 1 \end{bmatrix}$$

$$\approx \begin{bmatrix} 0.816 \\ 0.408 \\ 0.408 \end{bmatrix}$$

$$\mathbf{u}_2 = \frac{1}{1} B \mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 \\ -1 \\ 1 \end{bmatrix}$$

Step 4. U is 3×3 and needs a third column, perpendicular to both. Try $[1, -1, -1]$:

$$[1, -1, -1] \cdot [2, 1, 1] = 2 - 1 - 1 = 0$$

$$[1, -1, -1] \cdot [0, -1, 1] = 0 + 1 - 1 = 0$$

Both dot products are 0, so

$$\mathbf{u}_3 = \frac{1}{\sqrt{3}} \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix}$$

The vector $\mathbf{u}_3$ spans the left null space, the one direction in 3D that B never reaches (the red arrow of Figure 8).

The other route, eigenvectors of BB^T , would mean a 3×3 eigenvalue problem (eigenvalues 3, 1 and 0), and would still risk the sign trap of Section 4. For a tall data matrix with thousands of rows, $A^T A$ is the small one, so we always start from it.

8. How computers compute the SVD

Key point: Forming $A^T A$ squares the singular values, and small ones get lost in rounding. Libraries compute the SVD directly from A .

In plain words. A computer keeps only about 16 significant digits of each number. Squaring a tiny number makes it far tinier, and next to a big number it falls off the end of those 16 digits.

On numbers.

a small singular value: 10^{-9}

its square, an eigenvalue of $A^T A$: 10^{-18}

next to an eigenvalue of 4, the ratio is $10^{-18}/4$

$$\approx 2.5 \times 10^{-19}$$

That is far below the 16th digit, so the small eigenvalue disappears in rounding.

Python: A nearly rank-1 matrix: its second singular value is tiny but not zero.

```
M = np.array([[1, 1], [1, 1 + 1e-9]])
np.linalg.svd(M)[1]           # [2.0, 5.0e-10]: correct
np.sqrt(np.linalg.eigvalsh(M.T @ M))
# [0.0, 2.0]: the small one is lost
```

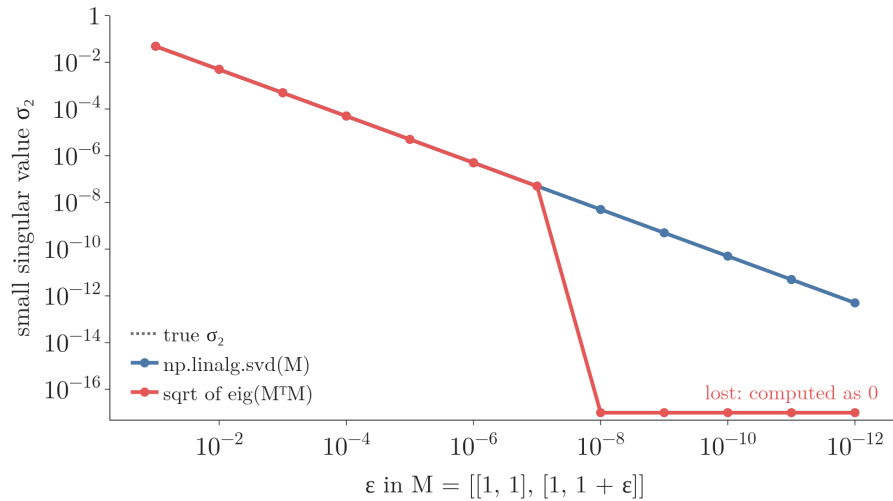


Figure 9: The small singular value of M as ϵ shrinks: `np.linalg.svd` (blue) follows the true value (dotted, under the blue line); the square root of the eigenvalue of $M^T M$ (red) drops to 0 from $\epsilon = 10^{-8}$ on

Figure 9 repeats the box for twelve values of the small number ϵ (the 10^{-9} in the code). Watch the red line: it agrees with the blue one while σ_2^2 still fits in the 16 digits next to $\sigma_1^2 = 4$, then falls off a cliff. So `np.linalg.svd` never forms $A^T A$; NumPy calls the LAPACK routine `gesdd` (NumPy docs, `numpy.linalg.svd`). That routine reduces A itself step by step with orthogonal matrices, which do not magnify rounding errors, until the singular values can be read off (Trefethen and Bau, Lecture 31).

Extra: The ratio of the largest to the smallest singular value is the **condition number** (G-441) of a matrix:

$$\text{condition number} = \frac{\sigma_1}{\sigma_n}$$

For A :

$$\frac{6.71}{2.24} = 3$$

It measures how much errors in the input can be magnified by solving with that matrix. Forming $A^T A$ squares it: a condition number of 10^8 becomes 10^{16} , which uses up all 16 digits (Trefethen and Bau, Lectures 12 and 19). The squaring is why the least-squares solvers of the [multiple linear regression code Note](#) (section 6) prefer to work from X itself rather than from $X^T X$. `np.linalg.cond(A)` computes it.

9. Summary

Step	What we compute	For A (rows $[3, 0]$, $[4, 5]$)	For C (rows $[2, 1]$, $[4, 2]$)
1	$A^T A$	rows $[25, 20]$, $[20, 25]$	rows $[20, 10]$, $[10, 5]$
2	eigenvalues, then $\sigma_i = \sqrt{\lambda_i}$	45, 5 give 6.708, 2.236	25, 0 give 5, 0
2	eigenvectors $\mathbf{v}_i$	$\frac{1}{\sqrt{2}}[1, 1]$, $\frac{1}{\sqrt{2}}[-1, 1]$	$\frac{1}{\sqrt{5}}[2, 1]$, $\frac{1}{\sqrt{5}}[-1, 2]$
3	$\mathbf{u}_i = A\mathbf{v}_i/\sigma_i$	$\frac{1}{\sqrt{10}}[1, 3]$, $\frac{1}{\sqrt{10}}[-3, 1]$	$\frac{1}{\sqrt{5}}[1, 2]$
4	complete U	nothing missing	$\frac{1}{\sqrt{5}}[-2, 1]$

- $A^T A = V\Sigma^T \Sigma V^T$: eigenvectors are the $\mathbf{v}_i$, eigenvalues the σ_i^2 .
- $A^T A$ is always symmetric and positive semi-definite, so this always works.
- Get the $\mathbf{u}_i$ from $A\mathbf{v}_i/\sigma_i$, not from a separate eigen-problem: that keeps the signs matched.
- Zero singular values belong to the null space; the missing $\mathbf{u}$'s are any perpendicular completion.
- The SVD gives perpendicular bases for the row space, null space, column space and left null space.
- Computers compute the SVD from A directly, because $A^T A$ loses small singular values.

10. Sources

Built from

- 3Blue1Brown (Sanderson, G.), “Inverse matrices, column space and null space | Chapter 7, Essence of linear algebra”, YouTube, <https://www.youtube.com/watch?v=uQhTuRIWMxw>
- Khan Academy (Khan, S.), “Column space of a matrix”, YouTube, <https://www.youtube.com/watch?v=v...>
- Khan Academy (Khan, S.), “Rowspace and left nullspace”, YouTube, <https://www.youtube.com/watch?v=...>
- Strang, G., MIT OpenCourseWare 18.06 *Linear Algebra*, “Lecture 29: Singular value decomposition” (video, recorded 1999), ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/resources/lecture-29-singular-value-decomposition. The recipe, the sign trap, the rank-1 example and the four subspaces (Sections 3 to 6).

Other references

- Strang, G. (2016). *Introduction to Linear Algebra*, 5th ed. Wellesley-Cambridge Press. Section 7.2, bases and matrices in the SVD: the recipe and the non-square example, which no video in the list works through.
- Deisenroth, M. P., Faisal, A. A. and Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. Section 4.5.2 (MML).
- NumPy documentation. `numpy.linalg.svd` (uses LAPACK `gesdd`).
- Trefethen, L. N. and Bau, D. (1997). *Numerical Linear Algebra*. SIAM. Lecture 12 (conditioning), Lecture 19 (least squares and the normal equations), Lecture 31 (computing the SVD).

11. Key terms

Term	Meaning
$A^T A$	The symmetric, positive semi-definite matrix whose eigenvectors are the right singular vectors and whose eigenvalues are the squared singular values
Row space	The span of the rows of a matrix; spanned by the $\mathbf{v}_i$ with $\sigma_i > 0$
Null space	All vectors a matrix sends to $\mathbf{0}$; spanned by the $\mathbf{v}_i$ with $\sigma_i = 0$
Column space	The span of the columns: every possible output; spanned by the $\mathbf{u}_i$ with $\sigma_i > 0$
Left null space	The output directions perpendicular to every column; spanned by the remaining $\mathbf{u}_i$
Rank (of a matrix)	The number of dimensions of the column space: how many dimensions the outputs fill
Four fundamental subspaces	Row space, null space, column space and left null space of a matrix
Condition number	σ_1/σ_n : how much a matrix can magnify errors when we solve with it